

Python SMTP 邮件发送指南

本文档介绍 SMTP 协议基础概念，以及如何用 Python 发送邮件，从最简单的脚本到完整的生产级实现。

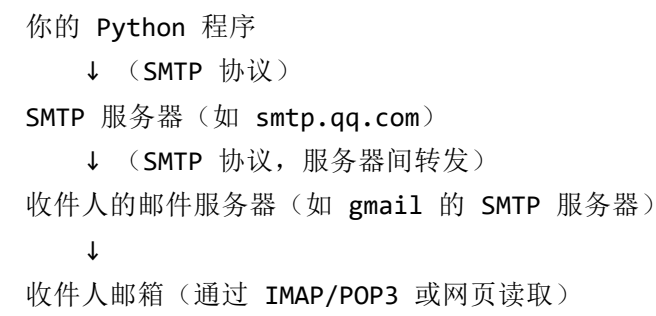
一、什么是 SMTP？

基本概念

SMTP（Simple Mail Transfer Protocol，简单邮件传输协议）是互联网上发送电子邮件的标准协议，定义在 RFC 5321 中。

一句话理解：SMTP 就是「寄信的邮局」——你的 Python 程序把邮件交给 SMTP 服务器，SMTP 服务器负责把信送到收件人的邮箱。

邮件发送的完整链路



关键术语

术语	说明	示例
SMTP 服务器	负责发送邮件的服务器	<code>smtp.qq.com</code> , <code>smtp.gmail.com</code>
IMAP/POP3 服务器	负责接收/读取邮件的服务器	<code>imap.qq.com</code> , <code>pop.gmail.com</code>
端口	SMTP 通常用 587（TLS）或 465（SSL）	587, 465

术语	说明	示例
授权码	邮箱服务商生成的专用密码，替代真实密码	16 位随机字符串
MIME	多用途互联网邮件扩展，支持附件、HTML 等	multipart/mixed
TLS/SSL	加密传输，保护邮件内容不被窃听	STARTTLS, SSL

为什么需要「授权码」而非邮箱密码？

现代邮箱服务商（QQ、网易、Gmail 等）出于安全考虑，**不允许直接用邮箱密码通过 SMTP 发邮件**。你需要：

- 1. 在邮箱设置中开启「SMTP 服务」
- 2. 生成一个**授权码**（专用密码，可以随时撤销）
- 3. Python 程序使用 **邮箱地址 + 授权码** 登录 SMTP 服务器

这就像给快递员一把专用钥匙，而不是你家的主钥匙。

二、准备工作（获取 SMTP 凭证）

QQ 邮箱（国内最常用）

- 1. 登录 QQ 邮箱网页版 → **设置** → **账户**
- 2. 找到「POP3/IMAP/SMTP/Exchange/CardDAV/CalDAV 服务」
- 3. 开启 **IMAP/SMTP 服务**
- 4. 按提示用手机发短信验证
- 5. 获得 **16 位授权码**（只显示一次，务必保存）

SMTP 服务器：smtp.qq.com
端口：465（SSL）或 587（STARTTLS）
用户名：your_qq@qq.com
密码：上面获得的授权码

163 网易邮箱

SMTP 服务器: `smtp.163.com`

端口: 465 (SSL) 或 994 (STARTTLS)

开启方式类似: 设置 → POP3/SMTP → 开启 → 获取授权码。

Gmail (需要梯子或特殊配置)

SMTP 服务器: `smtp.gmail.com`

端口: 587 (STARTTLS)

Gmail 需要在 Google 账户设置中开启「应用专用密码」(需先开启两步验证)。

QQ 企业邮箱

SMTP 服务器: `smtp.exmail.qq.com`

端口: 465 (SSL)

三、Python 内置库实现

Python 标准库 `smtplib` + `email` 即可实现邮件发送, 无需安装任何第三方包。

3.1 最简示例：发送纯文本邮件

```
import smtplib
from email.mime.text import MIMEText

# ===== 配置 =====
SMTP_SERVER = "smtp.qq.com"
SMTP_PORT = 465          # SSL 端口
SENDER_EMAIL = "your_qq@qq.com"
AUTH_CODE = "your_16_char_auth_code"  # 授权码，不是邮箱密码
RECEIVER_EMAIL = "recipient@example.com"

# ===== 构建邮件 =====
msg = MIMEText("这是一封测试邮件，由 Python 发送。", "plain", "utf-8")
msg["From"] = SENDER_EMAIL
msg["To"] = RECEIVER_EMAIL
msg["Subject"] = "Python SMTP 测试"

# ===== 发送邮件 =====
with smtplib.SMTP_SSL(SMTP_SERVER, SMTP_PORT) as server:
    server.login(SENDER_EMAIL, AUTH_CODE)
    server.sendmail(SENDER_EMAIL, RECEIVER_EMAIL, msg.as_string())

print("邮件发送成功！")
```

3.2 发送 HTML 格式邮件

```
import smtplib
from email.mime.text import MIMEText

html_content = """
<html>
<body>
    <h2 style="color: #4A90D9;">你好! </h2>
    <p>这是一封 <b>HTML 格式</b> 的邮件。</p>
    <table border="1" cellpadding="8" style="border-collapse: collapse;">
        <tr><th>项目</th><th>状态</th></tr>
        <tr><td>任务 A</td><td style="color:green;">✅ 完成</td></tr>
        <tr><td>任务 B</td><td style="color:orange;">⌚ 进行中</td></tr>
        <tr><td>任务 C</td><td style="color:red;">❌ 未开始</td></tr>
    </table>
    <p>详情请访问 <a href="https://example.com">这里</a>。</p>
</body>
</html>
"""

msg = MIMEText(html_content, "html", "utf-8")
msg["From"] = "your_qq@qq.com"
msg["To"] = "recipient@example.com"
msg["Subject"] = "HTML 邮件测试"

with smtplib.SMTP_SSL("smtp.qq.com", 465) as server:
    server.login("your_qq@qq.com", "your_auth_code")
    server.sendmail("your_qq@qq.com", "recipient@example.com", msg.as_string())
```

3.3 发送带附件的邮件

附件需要用 `MIMEMultipart` 构建多部分邮件：

```

import smtplib
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from email.mime.application import MIMEApplication

# ===== 配置 =====
SMTP_SERVER = "smtp.qq.com"
SMTP_PORT = 465
SENDER = "your_qq@qq.com"
PASSWORD = "your_auth_code"
RECEIVER = "recipient@example.com"

# ===== 构建多部分邮件 =====
msg = MIMEMultipart()
msg["From"] = SENDER
msg["To"] = RECEIVER
msg["Subject"] = "带附件的邮件"

# 正文
body = """
<p>你好, </p>
<p>附件中包含本次报告的数据文件, 请查收。</p>
"""

msg.attach(MIMEText(body, "html", "utf-8"))

# 添加附件 (文件路径)
attachment_path = r"C:\Users\lanpi\report.xlsx"

with open(attachment_path, "rb") as f:
    attachment = MIMEApplication(f.read(), Name="report.xlsx")
    attachment["Content-Disposition"] = f'attachment; filename="report.xlsx"'
    msg.attach(attachment)

# ===== 发送 =====
with smtplib.SMTP_SSL(SMTP_SERVER, SMTP_PORT) as server:
    server.login(SENDER, PASSWORD)
    server.sendmail(SENDER, RECEIVER, msg.as_string())

print("带附件的邮件发送成功!")

```

3.4 多个附件 + 多个收件人

```
import smtplib
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from email.mime.application import MIMEApplication
from email.utils import formataddr
```

```
def send_email_with_attachments(
    sender, password,
    to_list, cc_list=None, bcc_list=None,
    subject, body, body_type="html",
    attachment_paths=None,
    smtp_server="smtp.qq.com", smtp_port=465,
    sender_name=None
```

```
):
    """
    发送邮件（支持多个收件人、抄送、密送、多个附件）
```

Args:

```
    sender: 发件人邮箱
    password: SMTP 授权码
    to_list: 收件人列表 ["a@example.com", "b@example.com"]
    cc_list: 抄送列表（可选）
    bcc_list: 密送列表（可选）
    subject: 邮件主题
    body: 邮件正文
    body_type: "html" 或 "plain"
    attachment_paths: 附件路径列表（可选）
    smtp_server: SMTP 服务器
    smtp_port: SMTP 端口
    sender_name: 发件人显示名称（可选）
    """
```

```
msg = MIMEMultipart()
```

```
# 发件人
```

```
if sender_name:
    msg["From"] = formataddr((sender_name, sender))
else:
    msg["From"] = sender
```

```
# 收件人 / 抄送 / 密送
```

```
msg["To"] = ", ".join(to_list)
```

```

if cc_list:
    msg["Cc"] = ", ".join(cc_list)
if bcc_list:
    msg["Bcc"] = ", ".join(bcc_list)

msg["Subject"] = subject

# 正文
msg.attach(MIMEText(body, body_type, "utf-8"))

# 附件
if attachment_paths:
    for path in attachment_paths:
        with open(path, "rb") as f:
            filename = path.split("/)[-1].split("\\")[-1]
            attachment = MIMEApplication(f.read(), Name=filename)
            attachment["Content-Disposition"] = f'attachment; filename="{filename}"'
            msg.attach(attachment)

# 所有收件人（sendmail 需要完整列表）
all_recipients = to_list[:]
if cc_list:
    all_recipients.extend(cc_list)
if bcc_list:
    all_recipients.extend(bcc_list)

# 发送
with smtplib.SMTP_SSL(smtp_server, smtp_port) as server:
    server.login(sender, password)
    server.sendmail(sender, all_recipients, msg.as_string())

# ===== 使用示例 =====
send_email_with_attachments(
    sender="your_qq@qq.com",
    password="your_auth_code",
    to_list=["alice@example.com", "bob@example.com"],
    cc_list=["manager@example.com"],
    subject="周报 - 第 25 周",
    body="""
<h2>周报摘要</h2>
<p>本周完成了 3 个主要任务，详见附件。</p>
""",
    attachment_paths=[

```



```
        r"C:\Users\lanpi\report_week25.pdf",
        r"C:\Users\lanpi\data_week25.xlsx",
    ],
    sender_name="小张"
)
```

四、端口和加密方式对比

端口	加密方式	连接方式	常用场景
465	SSL（连接即加密）	smtplib.SMTP_SSL()	QQ、163 等（推荐）
587	STARTTLS （先明文再升级加密）	smtplib.SMTP() + starttls()	Gmail、国际服务
25	无加密	smtplib.SMTP()	服务器间转发 （ 不推荐 用于客户端）

587 端口的写法（STARTTLS）

```
with smtplib.SMTP("smtp.gmail.com", 587) as server:
    server.starttls() # 先连接，再升级为加密
    server.login("your@gmail.com", "app_password")
    server.sendmail(...)
```

五、email 库核心模块速查

Python 的 email 包由多个模块组成，各司其职：

```
email
├── email.mime          # 构建 MIME 邮件（最常用）
│   ├── MIMEText       # 文本/HTML 正文
│   ├── MIMEApplication # 二进制附件（任意文件）
│   ├── MIMEMultipart  # 多部分邮件容器（附件必须用这个）
│   ├── MIMEImage      # 图片附件（内嵌）
│   └── MIMEBase        # 自定义 MIME 类型
├── email.header        # 处理非 ASCII 主题/发件人名
├── email.utils          # 工具函数（formataddr、parsedate 等）
└── email.parser         # 解析收到的邮件
```

常用组合

纯文本邮件

```
msg = MIMEText("内容", "plain", "utf-8")
```

HTML 邮件

```
msg = MIMEText("<h1>标题</h1>", "html", "utf-8")
```

带附件的邮件

```
msg = MIMEMultipart()
```

```
msg.attach(MIMEText("正文", "html", "utf-8")) # 正文
```

```
msg.attach(MIMEApplication(data, Name="file.pdf")) # 附件
```

发件人显示中文名

```
msg["From"] = formataddr(("张三", "zhangsan@qq.com"))
```

六、常见错误与解决方案

错误 1： `smtpplib.SMTPAuthenticationError: (535, b'Login Fail')`

原因：授权码错误，或未开启 SMTP 服务。

解决：

1. 确认已开启邮箱的 SMTP 服务
2. 确认使用的是**授权码**，不是邮箱登录密码
3. 注意授权码中不要有多余的空格

错误

2:

smtpplib.SMTPServerDisconnected: Connection unexpectedly closed

原因：端口或加密方式不匹配。

解决：

- QQ 邮箱用 **465 + SSL** (SMTP_SSL)
- Gmail 用 **587 + STARTTLS** (SMTP + starttls())
- 不要在 SSL 端口上用 SMTP() ，也不要 STARTTLS 端口上用 SMTP_SSL()

错误 3：中文附件名乱码

解决：使用 email.header 编码文件名：

```
from email.header import Header
from email.mime.application import MIMEApplication

attachment = MIMEApplication(data, Name=Header("报告.xlsx", "utf-8").encode())
```

错误 4： ConnectionRefusedError: [Errno 111] Connection refused

原因：服务器地址或端口错误，或网络不通。

解决：确认 SMTP 服务器地址和端口正确，测试网络连通性：

```
import socket
socket.create_connection(("smtp.qq.com", 465), timeout=5) # 测试连接
print("连接成功")
```

错误

5:

error: [Errno 10054] An existing connection was forcibly closed

原因：被邮箱服务商限流或封禁（发送太频繁）。

解决：

- 降低发送频率（加入 time.sleep() ）

- 检查是否被标记为垃圾邮件发送者
- QQ 邮箱对每日发送量有限制（约 500 封/天）

七、进阶：第三方库（简化开发）

虽然标准库足够强大，但第三方库可以大幅简化代码：

7.1 yagmail（最简洁）

```
# pip install yagmail
import yagmail

yag = yagmail.SMTP("your_qq@qq.com", "your_auth_code", host="smtp.qq.com", port=465)
yag.send(
    to="recipient@example.com",
    subject="测试",
    contents="正文内容",
    attachments=["report.pdf"], # 支持列表
)
```

7.2 python-email（功能全面）

```
# pip install python-email
from email_message import EmailMessage

email = EmailMessage()
email["From"] = "you@qq.com"
email["To"] = "them@example.com"
email["Subject"] = "测试"
email.body = "纯文本正文"
email.html = "<h1>HTML 正文</h1>"
email.attach("report.pdf")

email.send(host="smtp.qq.com", port=465, username="you@qq.com", password="auth_code")
```

标准库 vs 第三方库

对比项	smtplib (标准库)	yagmail	python-email
安装	无需	pip install	pip install
代码量	较多	极简	中等
附件处理	手动 MIME	一行搞定	一行搞定
HTML 邮件	手动 MIME	自动	自动
适用场景	生产环境、完全控制	快速原型、简单需求	平衡方案

八、安全注意事项

⚠ 不要把授权码硬编码在代码中

```
# ❌ 危险：授权码直接写在代码里
PASSWORD = "abcdefghijklmnop"

# ✅ 安全：从环境变量读取
import os
PASSWORD = os.environ["SMTP_AUTH_CODE"]

# ✅ 安全：从配置文件读取（不提交到 Git）
# .env 文件（加入 .gitignore）
# SMTP_AUTH_CODE=abcdefghijklmnop
```

⚠ 不要用 25 端口

端口 25 是明文传输，邮件内容可被窃听。始终使用 **465 (SSL)** 或 **587 (STARTTLS)**。

⚠ 注意发送频率

大多数邮箱服务有每日发送上限（QQ ~500封/天，Gmail ~500封/天），超过会被临时封禁。批量发送时加入延时：

```
import time
for recipient in recipient_list:
    send_email(recipient)
    time.sleep(2) # 每封间隔 2 秒
```

九、快速上手清单

1. **[]** 登录邮箱网页版 → 开启 SMTP 服务 → 获取授权码
2. **[]** 用 3.1 最简示例测试纯文本发送
3. **[]** 用 3.2 示例测试 HTML 邮件
4. **[]** 用 3.3 示例测试带附件
5. **[]** 把授权码存到环境变量（不硬编码）
6. **[]** 根据需求选择标准库或第三方库